

HDOC Day-23

Menu-Driven Pattern Program in C

Name	Yug Nirwal	SAP ID	590033354
Subject	Programming for Problem Solving	Date	16 September 2026

Problem Statement

Write a menu-driven C program using a switch statement to print four basic square/rectangular patterns. Use nested loops without arrays. Each pattern is implemented in a separate parameterized void function. Prepare the algorithm and test cases, then compile, execute and test the program.

Algorithm

1. Start the program.
2. Declare choice, m and n.
3. Display the menu with four pattern options and Exit.
4. Read the user choice.
5. Use a switch statement to select the required pattern.
6. For square, read n and call solidSquare(n).
7. For rectangle, row-number and column-number patterns, read m and n and call the corresponding function.
8. Inside each function, use nested loops to print the required pattern.
9. Repeat the menu until the user selects Exit.
10. Stop.

Test Case Table

Test	Choice / Input	Expected Result	Actual Result	Status
1	1, n = 4	4 x 4 solid square	4 x 4 solid square	Pass
2	2, m = 3, n = 5	3 x 5 solid rectangle	3 x 5 solid rectangle	Pass
3	3, m = 4, n = 4	Each row contains its row number	Matches expected pattern	Pass
4	4, m = 4, n = 4	Each row prints 1 2 3 4	Matches expected pattern	Pass
5	5	Program exits	Program exits	Pass

C Program

```
#include <stdio.h>
```

```
void solidSquare(int n) {  
    int i, j;
```

```

        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++)
                printf("* ");
            printf("\n");
        }
    }

void solidRectangle(int m, int n) {
    int i, j;
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void rowNumber(int m, int n) {
    int i, j;
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", i);
        printf("\n");
    }
}

void columnNumber(int m, int n) {
    int i, j;
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", j);
        printf("\n");
    }
}

int main() {
    int choice, m, n;
    do {
        printf("\n1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter number of rows: ");
                scanf("%d", &n);
                solidSquare(n);
                break;
            case 2:
                printf("Enter rows and columns: ");
                scanf("%d %d", &m, &n);
                solidRectangle(m, n);
                break;
            case 3:
                printf("Enter rows and columns: ");
                scanf("%d %d", &m, &n);
                rowNumber(m, n);
                break;
            case 4:
                printf("Enter rows and columns: ");
                scanf("%d %d", &m, &n);
                columnNumber(m, n);
                break;
            case 5:
                printf("Exiting program...\n");

```

```

        break;
    default:
        printf("Invalid choice. Try again.\n");
    }
} while (choice != 5);
return 0;
}

```

Compilation and Execution

Save the program as day23.c. In Kali Linux, compile and run it using:

```
gcc day23.c -o day23
./day23
```

The program was compiled successfully and tested with the prepared test cases.

Test Case Execution Screenshots

Test Case 1

```

(kali~kali)-[~/Desktop/cprog]
└─$ ./a.out
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter number of rows: 4
* * * *
* * * *
* * * *
* * * *

```

Test Case 2

```

(kali~kali)-[~/Desktop/cprog]
└─$ ./a.out
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter rows and columns: 3 5
* * * * *
* * * * *
* * * * *

```

Test Case 3

```
(kali[kali])-[~/Desktop/cprog]
└─$ ./a.out
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 3
Enter rows and columns: 4 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

Test Case 4

```
(kali[kali])-[~/Desktop/cprog]
└─$ ./a.out
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 4
Enter rows and columns: 4 4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

Test Case 5

```
(kali[kali])-[~/Desktop/cprog]
└─$ ./a.out
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 5
Exiting program...
```

Result

The menu-driven C program was compiled and executed successfully. All four required patterns were printed correctly for the tested inputs, and all prepared test cases passed.